

Results

Results I

template/ was evaluated through multi-project pipeline execution, measuring test coverage, pipeline timing, output integrity, and steganographic performance across the canonical exemplars under projects/.

Multi-Project Pipeline Execution I

Runs used the `./run.sh` interactive orchestrator (“all projects core (fast)” / menu key `d`) skipping infrastructure replication and optional LLM stages orchestrated via `python -m infrastructure.orchestration`. **Note:** lone menu `d` returns after success without redrawing the TUI banner.

Project	Effective core stages ¹	Approx. duration	Tests ²	Coverage
template_cod	8	~40 s	196/196	90%+
e_project				
template_pro	8	~35 s	76/76	90%+
se_project				
template_aut	8	~30 s	151/151	90%+
oresearch_pr				
oject				

Multi-Project Pipeline Execution II

¹“Core-only” disables LLM-tagged YAML stages; durations exclude optional network-heavy LLM or long-running retrieval scripts when run with cached fixtures.

²Counts show passing tests versus discovered tests for the sampled configuration.

³`template_search_project` lives under `projects_archive/` (local-only literature-search exemplar); it is not part of the public CI roster.

Overall success: 100 % pipeline completion for sampled runs.

Timing illustrative (Apple Silicon workstation, SSD, fixed seeds).

Search-stage overhead dwarfs the optimization exemplar's runtime—confirming Stage 02 remains the bottleneck for outbound API traffic while retaining Zero-Mock subprocess + filesystem checks downstream.

Infrastructure Test Suite I

Metric	Value
Test files	367+
Total tests	~7,024
Infrastructure coverage gate	60 % (repo 80 %+ during recent audits)
Zero-mock imports	Verified via static scanning

Infrastructure Test Suite II

Exercises Pandoc/XeLaTeX paths, steganography hashing, telemetry, YAML-driven pipeline DAG resolution, HTTPS-bound optional suites (`pytest-httpserver`), and local Ollama-gated subsets.

Infrastructure Module Inventory I

The introspection module (`template.introspection`) emits the authoritative table below—every row reflects `discover_infrastructure_modules(REPO_ROOT)`.

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	7			<code>build_autoresearch_plan</code> , <code>readiness validation CLI</code>
benchmark	3			Template harness <code>scoring + comparative gates</code>
config	0			Repository defaults + hardened templates
core	97			<code>get_logger</code> , <code>load_config</code> , <code>TemplateError</code>
docker	0			Containerisation
doctor	9			scaffolding Checkout <code>diagnose/fix/undo repairs</code>
documentation	10			<code>FigureManager</code> , <code>generate_glossary</code>
llm	53			Ollama helpers, <code>sanitization</code> , <code>review + translation pipelines</code>
logrotate.d	0			Rotation snippets (documentation-first)
methods	5			<code>build_methods_orchestration_plan</code> , <code>methods-stage contracts + validation</code>

Infrastructure Module Inventory II

orchestration	8	PipelineRunner, entry point for ./run.sh
project	19	discover_projects, workspace management
prose	8	Markdown readability + prose tooling
publishing	42	Zenodo, executable bundle, archival targets
reference	8	BibTeX models, parsers, converters
rendering	45	PDF/HTML/slide rendering, Pandoc filters
reporting	51	Coverage parsers, dashboards, executive artefacts
scientific	6	check_numerical_sta bility, benchmark_f unction
search	8	infrastructure.sear ch.literature clients + cache
skills	6	discover_skills, SKILL manifest regeneration
steganography	10	Watermark overlays + hash manifests
validation	61	PDF + Markdown + integrity CLIs

Infrastructure Module Inventory III

All 22 enumerated subdirectories carry Tier-1/README.md and Tier-2/AGENTS.md coverage wherever the Documentation Duality standard applies; subsets ship Tier-3 SKILL.md descriptors for MCP routing (infrastructure/skills manifest generation).

Agentic Skill Documentation Coverage I

Layer	Role
System prompts	Root CLAUDE.md, README policy
Structural	AGENTS.md directories
Skills	Optional SKILL.md manifests + generated .cursor/skill_manifest.json
PAI capsule	Repository level PAI.md narratives

Agentic Skill Documentation Coverage II

351+ Markdown shards under docs/ capture operational knowledge without duplicating auto-generated inventories.

DAG Reference (Declarative Executor) I

Stages below mirror pipeline.yaml (executor-topological order—not strict numeric script filenames). Scripts live under scripts/.

Name	Typical script / method	Responsibility	Failure semantics
Clean Output Directories Environment Setup Infrastructure Tests Project Tests Project Analysis PDF Rendering Output Validation LLM Scientific Review LLM Translations Copy Outputs	<code>_run_clean_outputs</code> <code>00_setup_environment.py</code> <code>01_run_tests.py --infra-only</code> <code>01_run_tests.py --project-only</code> <code>02_run_analysis.py</code> <code>03_render_pdf.py</code> <code>04_validate_output.py</code> <code>06_llm_review.py --reviews-only</code> <code>06_llm_review.py --translations-only</code> <code>05_copy_outputs.py</code>	Deletes stale output/ trees Validates tooling, PYTHONPATH scaffolding Infra pytest + coverage gates Project pytest + coverage gates Executes projects/<name>/scripts/*.py Pandoc → XeLaTeX manuscripts Structural PDF/markdown probes Local Ollama reviews Optional translations Mirrors deliverables → output/<project>/	Blocking Blocking Tunable thresholds Zero failures default Blocking Blocking Blocking / warnings Skippable / exit 2 tolerated Skippable Soft-fail surfaced in logs

DAG Reference (Declarative Executor) II

`scripts/07_generate_executive_report.py` is **multi-project orchestration glue** invoked after iterating active projects—not a tenth DAG node for single-repo runs (`execute_pipeline.py`).

Steganographic Performance I

Project	Pages (approx.)	Metadata	SHA-256	Overlay	QR Code	Total (approx.)
template_code_project	~20	<0.3 s	<0.05 s	<0.8 s	<0.4 s	<1.5 s
template_project	~25	<0.3 s	<0.05 s	<0.9 s	<0.4 s	<1.6 s
template_automatedresearch_project	~25	<0.2 s	<0.04 s	<0.9 s	<0.3 s	<1.5 s

Steganographic Performance II

Measurements single-thread Apple Silicon; dominated by watermark overlay complexity.

Self-Referential Analysis I

Rendered via `projects_in_progress/template (generate_manuscript_metrics.py` → injected tokens such as 22).

Architecture figures stem from `template.architecture_viz`—font sizes constrained by §QA.

Self-Referential Analysis II

Two-Layer Architecture

Layer 1: Infrastructure (generic · reusable · cross-project)

Layer 2: Projects (domain-specific · customizable · isolated)

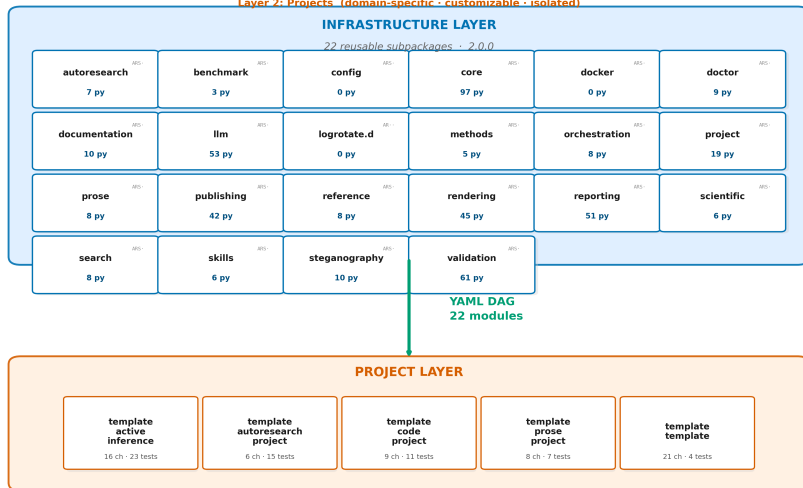


Figure 1. Live Two-Layer graph with documentation badges [ARSP] and per-module file counts derived from introspection snapshots.

Comparative Feature Analysis I

Figure 4 summarizes the Appendix F capability matrix.

Comparative Feature Analysis II

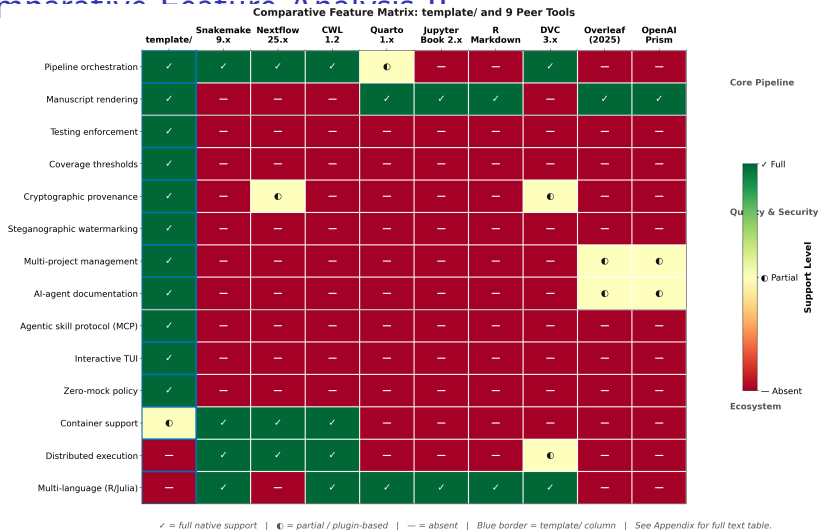


Figure 4. 14 × 10 heatmap annotated in appendix text—green full native capability, yellow partial / plugin-hosted, red — unavailable. Rows group *Core Pipeline*, *Quality & Security*, then *Ecosystem*.

Test Quality Metrics I

- ▶ **Zero mocks:** repository policy bans `unittest.mock` / patching frameworks in tests.
- ▶ **Filesystem + subprocess realism:** ephemeral directories + actual CLI binaries.
- ▶ **HTTP realism:** infra suites favour `pytest-httpserver`; literature tests hit recorded fixtures.
- ▶ `template_code_project` focuses on numerical reproducibility assertions.
- ▶ `template_autoresearch_project` exercises the AutoResearch readiness planner (`infrastructure/autoresearch/`). `template_search_project` remains archive-only for literature-search workflows.